



Sortable LTI 1.3 dashboard: Canvas vs Blackboard vs Moodle vs D2L Brightspace API limits, OAuth scopes, and AGS/NRPS endpoints compared

Computer

LMS migration runbook: Canvas vs Blackboard vs Moodle vs D2L Brightspace authentication, rate limits, and endpoint mapping in an audit report

Computer

Show the Blackboard and Moodle details

Compare the rate limits and pagination rules

Map out the shared object model schema

Blackboard and Moodle are the two platforms where Semester should use the most cautious, tenant-specific integration strategy. Both can support LTI 1.3 workflows, but their actual API behavior, enabled services, identity configuration, paging, quotas, and hosting model can vary more than Canvas or Brightspace.

The implementation standard should therefore be: **standards first, discovery second, proprietary APIs third**. Launch with LTI 1.3; use NRPS and AGS only when they appear in verified tenant launch/service configuration; use REST or Web Services only for a documented, approved need; and reconcile every important synchronization.

Blackboard Learn details

Integration architecture

```
Blackboard Learn tenant
├── LTI 1.3 / LTI Advantage platform services
│   ├── OIDC login initiation
│   ├── JWT launch
│   ├── Deep Linking, if enabled
│   ├── NRPS, if enabled
│   └── AGS, if enabled and authorized
├── Blackboard REST APIs
│   ├── Courses
│   ├── Users
│   ├── Enrollments
│   ├── Content
│   ├── Gradebook
│   └── Other enabled APIs
└── Blackboard administration
    ├── Application registration
    └── OAuth 2.0 credentials
```

- └─ LTI tool approval
- └─ Data/role configuration
- └─ Tenant-specific permissions

Blackboard LTI 1.3 checklist

- [] Confirm Learn deployment type:
SaaS, managed hosting, or self-hosted.
- [] Confirm exact Learn release/version.
- [] Register Semester as an LTI 1.3 tool.
- [] Obtain and store:
issuer
client ID
deployment ID
OIDC initiation endpoint
authorization endpoint
JWKS endpoint
OAuth access-token endpoint
redirect URI allowlist
- [] Confirm which Advantage services are enabled:
Deep Linking
Names and Role Provisioning Service
Assignment and Grade Services
- [] Confirm user identity fields released in the launch:
subject
name
email
roles
institution/person identifiers
custom claims
- [] Confirm grade-service permissions:
Create line item
Read line item
Post score
Read results
View grade status in student view
- [] Confirm role mapping:
Instructor
Teaching assistant
Student
Observer
Administrator
Custom institutional roles
- [] Confirm data retention and logging expectations.

Blackboard's developer portal exposes REST API endpoint requirements, request parameters, sample JSON, and response examples, and its REST API framework uses application

registration/OAuth 2.0. However, the actual API surface and behavior must be validated against the customer’s supported Learn version and hosting model. [1650][1648]

Blackboard API use cases

Use Blackboard REST APIs only when LTI does not cover the need.

Need	Prefer	REST API only when
Launch Semester in a course	LTI 1.3 Core	Not applicable
Identify current course context and role	LTI launch claims	Additional system context is essential
Read roster	NRPS	NRPS is unavailable or insufficient and customer approves REST scope
Create content link	Deep Linking	Customer requires a specific managed-placement workflow
Post an assessment score	AGS	Institution requires a Blackboard-specific official-grade workflow
Read course content	LTI/context and approved sources	Semester needs specific customer-approved content synchronization
Read grades	AGS results	Institution explicitly authorizes a Blackboard gradebook integration
Provision/deprovision accounts	SCIM/identity provider	Blackboard API is a defined customer-approved lifecycle source

Blackboard risk controls

Do not assume API parity across SaaS and non-SaaS deployments.

Do not assume every LTI Advantage service is enabled.

Do not treat Blackboard REST access as a substitute for proper LTI context.

Do not use instructor credentials for server-to-server integration.

Do not bulk-pull student content or grade data without purpose limitation.

Do not rely on undocumented request limits or event behavior.

Do not write grades without a preview, idempotency record, sync status, and reconciliation process.

Moodle details

Integration architecture

Moodle site

- └─ LTI 1.3 platform/tool support
 - | └─ OIDC login initiation
 - | └─ JWT launch
 - | └─ Advantage services where configured
 - | └─ Tool deployment/configuration
- └─ Moodle Web Services
 - | └─ REST
 - | └─ SOAP

- |
 - |— XML-RPC
 - |— Other enabled protocols
- |— Moodle plugins
 - |— Local/custom plugins
 - |— Authentication/enrollment plugins
 - |— Event observers
 - |— Reporting/custom data sources
- |— Moodle administration
 - |— Site version
 - |— Hosting model
 - |— Enabled web services
 - |— External services/functions
 - |— Token/user restrictions
 - |— Plugin inventory
 - |— Reverse-proxy/WAF/rate controls

Moodle LTI 1.3 checklist

- [] Confirm Moodle version and support lifecycle.
- [] Confirm hosting:
MoodleCloud, managed hosting, institutional self-hosting,
consortium/shared service, or customized deployment.
- [] Confirm whether Semester is configured as:
external tool / LTI consumer / LTI provider / both.
- [] Obtain and store:
 - issuer
 - client ID
 - deployment ID
 - OIDC initiation endpoint
 - authorization endpoint
 - JWKS endpoint
 - access-token endpoint
 - redirect URI allowlist
- [] Confirm LTI Advantage services enabled:
 - NRPS
 - AGS
 - Deep Linking
- [] Confirm course, user, role, and identity claims.
- [] Confirm allowed external web-service functions.
- [] Confirm authentication model:
 - service token
 - OAuth 2
 - plugin-managed flow
 - user-bound token
 - other approved institutional method.
- [] Confirm plugin inventory and customizations affecting:
 - roles, enrollments, gradebook, events, content, files, privacy,

retention, and API behavior.

[] Confirm test/staging site availability.

Moodle's developer resources document an OAuth 2 API for configuring/managing external OAuth 2 systems, while Moodle documentation describes site-admin OAuth 2 service configuration. Moodle's LTI behavior and Web Services exposure remain highly dependent on the deployed version, enabled services, plugins, and local administration. ^[1643][1645]

Moodle Web Services principles

Use a dedicated integration service account.
Grant only the specific external-service functions needed.
Restrict the service account to the smallest role/context possible.
Separate sandbox credentials from production credentials.
Rotate tokens and credentials.
Disable unused services/protocols.
Log function calls and export activity.
Use version-compatible functions and test after Moodle/plugin upgrades.
Avoid custom plugin dependency unless the customer can own its lifecycle.

Moodle plugin caution

Moodle's extensibility is valuable, but plugins can become a hidden integration liability.

Plugin risk review:

- Maintainer and update cadence
- Security advisories and CVE response
- Supported Moodle versions
- Data processing and subprocessors
- Role/capability changes
- Database schema impact
- Performance impact
- Privacy API support
- Backup/restore behavior
- Accessibility testing
- Exit/removal plan

Semester should prefer:

LTI 1.3
→ standard Moodle Web Services
→ approved reporting/export path
→ custom plugin only as a last resort

Rate limits and pagination

Comparison table

Dimension	Canvas	Blackboard Learn	Moodle	D2L Brightspace	Semester integration standard
Rate-limit model	Dynamic cost-based throttling with replenishing quota	Customer deployment/version-specific; confirm during onboarding	Instance/host/plugin/reverse-proxy-specific	Token-bucket credit model; variable call cost	Adaptive per-tenant worker limits and queue
Published fixed global limit	No universal simple RPM figure	No dependable universal production figure	No universal cross-installation figure	Published credit bucket mechanism; verify tenant policy	Never hardcode a vendor-wide RPM
Throttle signal	HTTP 429 and quota-related behavior	Must test 429/headers/errors in target tenant	Must test host behavior, 429/503/WAF response	HTTP 429 plus rate-limit headers and Retry-After	Parse headers, honor retry, record all throttles
Pagination style	Endpoint-specific pagination/Link headers and per-page parameters	Endpoint/version-specific paging parameters	Web-service-function-specific <code>limitfrom/limitnum</code> or custom pattern	Endpoint-specific page/bookmark/query model	Connector-specific paginator with checkpointing
Incremental sync	API/event strategy varies by resource	Validate modified-since/filter support by endpoint	Often needs timestamp/checkpoint/custom report approach	Validate endpoint filters/change feeds	Store watermark plus overlap window and reconcile
Bulk sync	Use queues, page sizes, cache, backoff	Stage/paginate, limited concurrency, reconcile	Throttle based on site capacity; schedule off-peak	Credits-aware queue, cache, limited concurrency	Resumable jobs with idempotent item processing
Event/webhook confidence	Tenant-specific capability; reconcile	Deployment-specific; reconcile	Plugin/customization dependent; reconcile	Tenant-specific; reconcile	Treat events as hints; source reconciliation is mandatory

Pagination rules

A robust Semester connector should never depend on page count alone.

1. Request the maximum documented safe page size only after tenant testing.
2. Persist the cursor, bookmark, Link-header URL, offset, or page token.
3. Store a checkpoint after every successful page.
4. Deduplicate records by stable external ID plus source version/updated time.
5. Use an overlap window for “modified since” queries to catch clock skew/delayed indexin
6. Use idempotent upserts rather than inserts.
7. Stop on 429, apply Retry-After/backoff, and resume from the checkpoint.
8. Place malformed/unreadable records in an exception queue; do not kill the full sync.

9. Run periodic full or scoped reconciliation.
10. Track count, lag, error rate, throughput, throttle events, and last good checkpoint.

Blackboard and Moodle discovery test

Before enabling a production connector, execute this controlled sandbox test:

Authentication:

- Obtain/refresh token.
- Test token expiry and key rotation.
- Remove a permission and verify failure is clear.

Pagination:

- Pull several pages of courses, enrollments, and content.
- Restart midway and resume from stored checkpoint.
- Verify no skipped/duplicated records after concurrent source changes.

Throttle:

- Run a bounded controlled burst.
- Record 429/503 behavior, rate-limit headers, Retry-After, error body, and recovery behavior.
- Set safe concurrency and sustained-call budget.

Write:

- Create a non-production artifact or grade line item if permitted.
- Repeat same request with idempotency record.
- Verify no duplicate write.
- Test revoke/rollback/reconciliation behavior.

Reconciliation:

- Introduce an intentional source-side change without an event.
- Confirm scheduled reconciliation catches and repairs the difference.

Shared object-model schema

The platform should model every module through shared, tenant-scoped objects. This prevents the LMS, study tools, clubs, career services, basic-needs navigator, and Operations Console from becoming disconnected applications.

Core hierarchy

Platform

- └─ Tenant / Institution
 - └─ Campus / Division
 - └─ Person
 - └─ Identity and account
 - └─ Role grant
 - └─ Academic term
 - └─ Course
 - └─ Community / organization
 - └─ Service / resource
 - └─ Opportunity
 - └─ Plan / scenario
 - └─ Task / action

- └─ Event
- └─ Document / source
- └─ Assessment / item / rubric
- └─ Submission / feedback / grade
- └─ Share / consent / referral
- └─ Integration connection
- └─ Notification
- └─ Conversation / support case
- └─ Audit event

Universal metadata envelope

Every major object should contain:

```
id
tenant_id
external_ids[]
object_type
owner_id
created_at
updated_at
created_by
updated_by

source_type
source_system
source_reference
source_url, where applicable
source_version
source_retrieved_at
authority_level

visibility_scope
access_policy_id
retention_class
retention_until
legal_hold_status

status
freshness_status
effective_at
expires_at
archived_at

version
previous_version_id
change_reason
audit_correlation_id

accessibility_metadata
language
rights_classification
data_classification
```

Core entities

Person and identity

Person

- id
- tenant_id
- display name fields
- contact methods
- profile visibility settings
- accessibility preference references
- lifecycle status

Identity

- id
- person_id
- provider: native / SAML / OIDC / LMS / SCIM
- provider_subject
- verified attributes
- authentication assurance
- linked_at / unlinked_at
- lifecycle state

RoleGrant

- id
- person_id
- scope_type: tenant / course / community / service / case
- scope_id
- role
- permissions
- granted_by
- granted_at
- expires_at
- status

Academic context

Term

- id
- tenant_id
- name
- start/end dates
- registration windows
- status

Course

- id
- tenant_id
- external_course_ids
- term_id
- title/code
- source system
- instructors
- status
- source freshness

```
Enrollment
- id
- course_id
- person_id
- role
- status
- start/end dates
- external enrollment ID
```

Sources and content

```
SourceDocument
- id
- tenant_id
- owner/sponsor
- title
- source type
- source URL/file reference
- rights/license
- course/community/service scope
- version
- review date
- access policy
- extraction/indexing policy
- retention class

SourceFact
- id
- source_document_id
- exact source anchor: page/section/timestamp
- structured claim/value
- confidence/extraction status
- reviewed by
- effective/expiry dates
```

Learning and assessment

```
Assignment
- id
- course_id
- prompt/source reference
- due date
- grading scheme
- rubric version
- AI/course policy
- status

AssessmentItem
- id
- item bank ID
- QTI version/package reference
- interaction type
- outcome mapping
- scoring rules
```

- accessibility review
- source/provenance
- version/status

Submission

- id
- assignment_id
- student/enrollment ID
- attempt number
- artifact references
- submitted_at
- receipt hash
- status
- integrity-review status

GradeEntry

- id
- submission_id
- rubric/version reference
- score/scale
- grader
- feedback references
- release status
- override/change reason
- external-sync status

Student actions and services

Task

- id
- person_id
- related object type/id
- title
- status
- due date
- source/authority
- visibility
- reminder policy

ServiceResource

- id
- tenant_id
- owner office/provider
- category
- eligibility
- cost
- availability
- access route
- accessibility/language data
- source/freshness
- privacy note

Referral

- id
- person_id
- service_resource_id
- data scope

- consent reference
- receiving office
- referral status
- created/expires dates

Trust, sharing, integration, and operations

ShareGrant

- id
- grantor
- grantee person/role/office
- object scope
- permissions
- purpose
- consent/legal basis
- issued/expires/revoked dates

IntegrationConnection

- id
- tenant_id
- provider
- capability tier
- credentials reference
- scopes
- source data map
- last sync / health / freshness
- error state
- owner
- offboarding state

Notification

- id
- recipient
- event type
- source object
- channel
- urgency
- preference decision
- delivery status
- audit reference

AuditEvent

- id
- tenant_id
- actor
- actor role
- action
- object type/id
- before/after reference
- purpose
- source IP/device metadata, minimized
- timestamp
- correlation ID

Universal trust API shape

Every user-facing object should expose the same trust payload:

```
{
  "source": {
    "type": "institution_verified",
    "system": "Blackboard Learn",
    "reference": "course_48933",
    "last_synced_at": "2026-09-28T14:30:00Z"
  },
  "authority": {
    "level": "official_course_record",
    "decision_owner": "Course Instructor"
  },
  "scope": {
    "visibility": "course_staff_and_student",
    "sharing": "not_shareable"
  },
  "status": {
    "state": "released",
    "freshness": "current",
    "effective_at": "2026-09-28T14:30:00Z"
  },
  "version": {
    "number": 4,
    "updated_at": "2026-09-28T14:30:00Z"
  }
}
```

Cross-domain relationship rules

A Course can link to Tasks, Events, Documents, Assignments, Assessments, Submissions, Feedback, Grades, Communities, Services, and Opportunities.

A Person controls private Plans, Tasks, Notes, Preferences, and eligible Shares.

A ServiceResource can create a Referral only through consent or a documented institutional basis.

A GradeEntry never becomes a career/employer data source by default.

Basic-needs browsing never becomes faculty/advisor/club visibility.

AI conversations never become a shared SourceDocument unless a user deliberately saves and shares an approved artifact.

IntegrationConnection data always retains source, freshness, and external ID.

Final implementation rule

Treat Blackboard and Moodle as **capability-discovery integrations**:

```
Register
→ validate LTI launch
→ discover released services/scopes
→ test pagination/rate behavior
→ enable only needed data flows
→ establish reconciliation
→ monitor health
→ revalidate after vendor/version/hosting change
```

Treat the shared object model as the product's permanent core. LMS systems may change; the Semester object model, trust envelope, design system, permissions, audit evidence, and source-aware user experience should remain consistent everywhere.



1. <https://moodledev.io/docs/5.0/apis>
2. https://docs.moodle.org/en/OAuth_2_authentication
3. <https://support.anthology.com/apex/downloadArticleasPDF?id=kaEPU0000004QVQ2A2>
4. <https://developer.blackboard.com/portal/displayApi>